Trust the State—Then Protect Every Byte

Project Hands-on Day 1: State Contract + Frame Integrity

`starter_repo` $\longrightarrow$ `frame_v1`

Prof. Dr. Mohammed Aquil Mirza – Jiang Suyu – Ruan Haozhe

July 2026 – Lanzhou University – PolyU

The starter repo is our only allowed starting point

Input `starter_repo` ⟶ **Output** `frame_v1`

Run once as a whole class

```
make smoke SEED=20260719
```

- Do not repair or reconfigure yet.
- On failure, restore the instructor bundle.

Record in `manifest.json`

- repository commit and toolchain version;
- fixed seed 20260719;
- supplied Linux-container image ID;
- smoke-test verdict and owner.

A missing unit and one flipped bit can target the wrong joint pose

Simulator-only failure: keep the unsafe path, then inject both faults.

`q[1]=90` unit omitted → consumer assumes radians → unchecked payload bytes → injector flips one bit → unchecked decode → wrong simulator joint target

Reproduce with the fixed seed

```
make fail-day1 CASE=unit-plus-bitflip
```

Record the signature

- assumed unit and first pose mismatch;
- flipped offset/mask and verdict;
- attempted actuation count.

Evidence target: `raw/day1_combined_failure.json`. The physical actuator remains disabled.

Trust the schema first; trust the frame only after integrity checks

Soda Code

```
const ARM_DOF: usize = 3;
struct ArmState {
  seq: u64, t_mono_ns: i64, frame_id: u8,
  q_rad: [f64; ARM_DOF], dq_rad_s: [f64; ARM_DOF],
  sigma_q_rad: [f32; ARM_DOF],
}
struct ArmCommand {
  seq: u64, t_mono_ns: i64, q_target_rad: [f64; ARM_DOF],
}
```

Freeze radians/metres/seconds, frame IDs, monotonic time, uncertainty, and joint limits.

Soda Code

```
use RxVerdict::*;
fn frame_accept(f: &FrameV1,
                last: u64) -> RxVerdict {
  if f.len != COMMAND_WIRE_LEN { NackLength }
  else if crc16(f) != f.crc { NackCrc }
  else if f.seq <= last { NackSequence }
  else { Accept }
}
```

Bound length, verify CRC, then reject duplicate/out-of-order sequence numbers.

Safety order: validate state → encode → verify frame → decode → simulate. No ACCEPT means no ACK and no actuation.

Today freezes both the payload and its protection boundary

State contract + fixed-step twin → **FrameV1** codec + gate → Transport + non-blocking runtime → Single writer + safety → Preemption + release evidence

May change today

- state validator and fixed-step twin;
- frame codec, CRC, parser, and resync;
- deterministic fixtures and fault injector.

Frozen after the gate

- DOF, units, frames, clock, joint limits;
- payload layout and wire version;
- ACCEPT/NACK meanings and baseline seed.

Fifteen students own ten chapters through five interfaces

Team	Build	Deliver	Pass
G1 Ch1–2	Freeze DOF, units, frames, time, uncertainty; run 3 known poses.	`state_v1` + chapter RQs	Same seeded trace; transform loop closes.
G2 Ch3–4	Wrap command; implement length/CRC/sequence/resync and corrupt fixtures.	`frame_v1` + fault table	Corrupt actuation = 0.
G3 Ch5–6	Freeze timestamped sensor/queue/write events and workload budget.	I/O contract + baseline trace	Every event uses one clock.
G4 Ch7–8	Define fail-closed verdict/reason semantics and a repeatable race fixture.	Safety contract + reject trace	One explicit verdict per fixture.
G5 Ch9–10	Connect controller adapter to `FrameV1` and the simulator gatekeeper.	Integration link v1 + audit	Clean path passes; corrupt path cannot actuate.

First trust the state; then protect its bytes

Complete only the marked functions

Soda Code

```
fn state_valid(s: &ArmState,
               prev: &ArmState) -> bool {
  // DAY1_TODO_A: finite, limits, frame,
  // sequence and monotonic time
  false
}
fn frame_feed(p: &mut Parser,
              b: u8) -> RxVerdict {
  // DAY1_TODO_B: sync, length, CRC,
  // sequence; ACK only after ACCEPT
  RxVerdict::NeedMore
}
```

`rg "DAY1_TODO" src tests`

Three roles, one branch

1. Chapter-A Lead: contract + 3 poses.
2. Chapter-B Lead: codec/parser + corrupt cases.
3. Evidence Lead: trace, manifest, interface check.

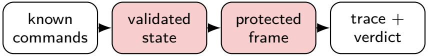
Milestones

- minute 11: `state_v1` checkpoint;
- minute 18: flip/truncate/reorder verdicts;
- minute 22: `make test-day1-gN`.

The gate requires deterministic state and zero corrupt execution

Run the class gate

```
make verify-day1
```



All five conditions must be green

- repeated seed gives a byte-identical state trace;
- 3 poses, transform loop, sequence, and time pass;
- decode(encode(trace)) preserves the payload;
- flip, truncation, and reorder get explicit verdicts;
- corrupted-frame actuation count = 0 for G1–G5.

Preserve the evidence pack
Baseline trace, corruption results, wire contract, and manifest; per chapter, keep 2–3 claim/method bullets and one caption.

Tomorrow, protected bytes must remain fresh in transit

`starter_repo` → `state_v1` internal checkpoint → **`frame_v1`** Day 2 input → `transport_v1` tomorrow's output

Create the tag only after a green gate

```
git tag -a frame_v1 -m "Day 1 green"
```

Handoff record

- owner, commit, seed, schema hash, wire version;
- unresolved corrupt case or `none`;
- matching last-green bundle.

Day 2 transports only accepted `FrameV1`; it does not weaken either Day 1 boundary.

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template